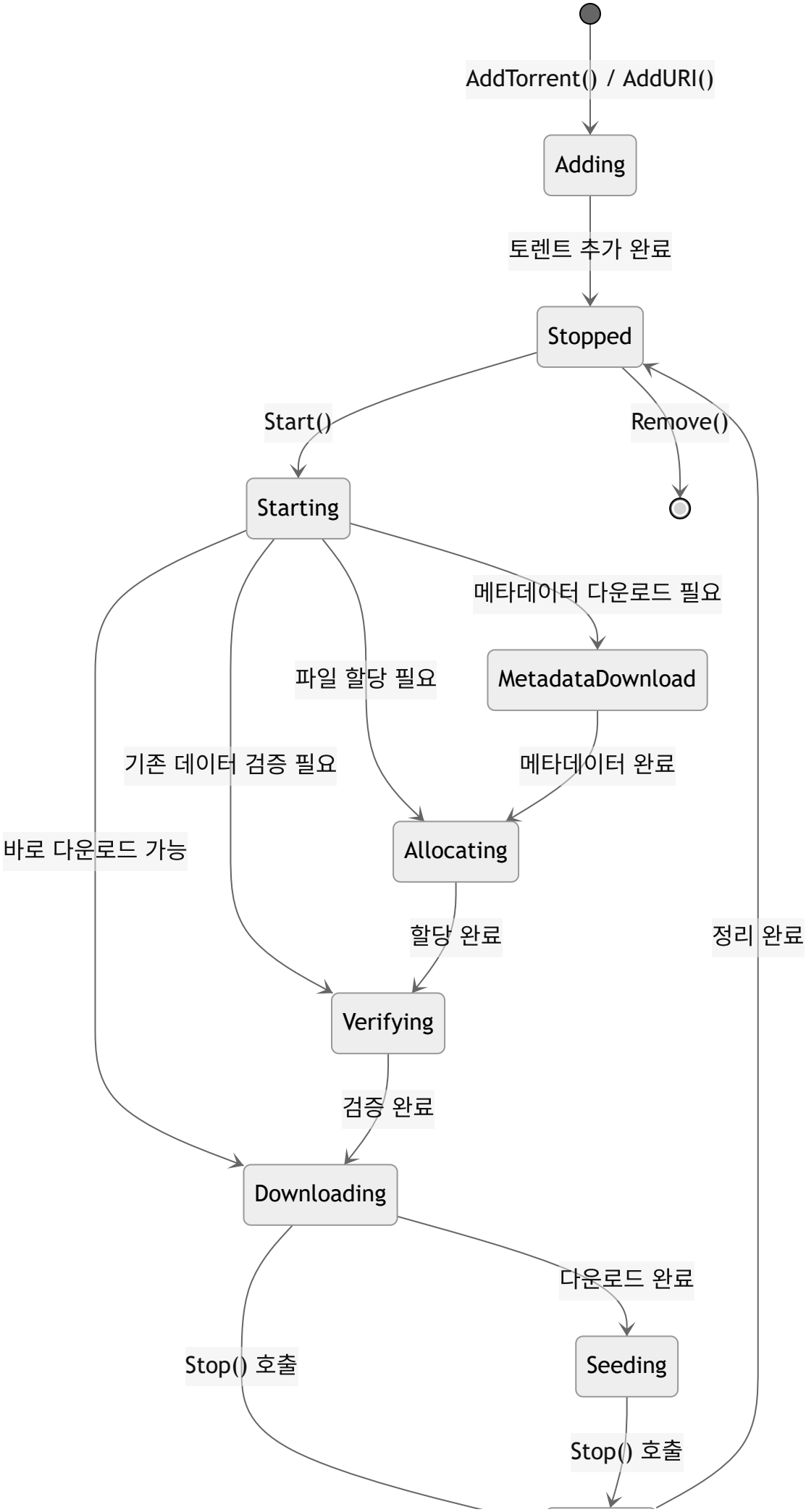
토렌트 라이프사이클 프로세스 분석

개요

이 문서는 Rain BitTorrent 클라이언트에서 토렌트의 전체 생명주기를 분석합니다. 토렌트 추가부터 완료까지의 모든 단계를 상세히 설명하며, 각 단계에서 호출되는 주요 함수와 프로세스를 다룹니다.

토렌트 라이프사이클 개요


AddTorrent() / AddURI()
Adding
토렌트 추가 완료
Stopped
Start()
Remove()
Starting
메타데이터 다운로드 필요
MetadataDownload
파일 할당 필요
기존 데이터 검증 필요
메타데이터 완료
바로 다운로드 가능
Allocating
할당 완료
정리 완료
Verifying
검증 완료
Downloading
다운로드 완료
Stop() 호출
Seeding
Stop() 호출

1. 토렌트 추가 프로세스

1.1 Session.AddTorrent() 함수 분석

`Session.AddTorrent()` 함수는 .torrent 파일에서 토렌트를 추가하는 메인 진입점입니다.

```
func (s *Session) AddTorrent(r io.Reader, opt *AddTorrentOptions) (*Torrent, err
if opt == nil {
opt = &AddTorrentOptions{}
}
t, err := s.addTorrentStopped(r, opt)
if err != nil {
return nil, err
}
if !opt.Stopped {
err = t.Start()
}
return t, err
}
```

주요 단계:

1. **옵션 검증**: `AddTorrentOptions` 구조체 초기화
2. **중지 상태로 추가**: `addTorrentStopped()` 호출

3. **자동 시작**: `opt.Stopped` 가 false인 경우 `Start()` 호출

1.2 addTorrentStopped() 내부 프로세스

```
func (s *Session) addTorrentStopped(r io.Reader, opt *AddTorrentOptions) (*Torre

// 1. 메타정보 파싱

r = io.LimitReader(r, int64(s.config.MaxTorrentSize))

mi, err := s.parseMetaInfo(r)

if  err != nil {

return  nil, newInputError(err)

}

// 2. 리소스 할당 (ID, 포트, 스토리지)

id, port, sto, err := s.add(opt)

if  err != nil {

return  nil, err

}

// 3. 토렌트 객체 생성

t, err := newTorrent2(...)

if  err != nil {

return  nil, err

}

// 4. 재시작 데이터 저장

rspec := &boltdbresumer.Spec{...}

err = s.resumer.Write(id, rspec)
```

```
// 5. 세션에 토렌트 등록
t2 := s.insertTorrent(t)
return t2, nil
}
```

핵심 구성요소:

- **메타정보 파싱**: `metainfo.New(r)` 로 .torrent 파일 파싱
- **포트 할당**: 사용 가능한 포트 범위에서 할당
- **스토리지 생성**: `storage.GetStorage(id)` 로 파일 저장소 생성
- **재시작 데이터**: BoltDB에 토렌트 상태 저장

1.3 Session.AddURI() 함수 분석

`AddURI()` 함수는 마그넷 링크나 HTTP URL에서 토렌트를 추가합니다.

```
func (s *Session) AddURI(uri string, opt *AddTorrentOptions) (*Torrent, error)
uri = filterOutControlChars(uri)
u, err := url.Parse(uri)
if err != nil {
return nil, newInputError(err)
}
switch u.Scheme {
case "http", "https":
return s.addURL(uri, opt)
```

```
case "magnet":
return s.addMagnet(uri, opt)
default:
return nil, errors.New("unsupported uri scheme: " + u.Scheme)
}
}
```

지원하는 URI 형식:

1. **HTTP/HTTPS URL**: 원격 .torrent 파일 다운로드
2. **마그넷 링크**: 메타데이터 없이 InfoHash로 시작

1.4 마그넷 링크 처리 (addMagnet)

마그넷 링크의 경우 메타데이터가 없으므로 특별한 처리가 필요합니다:

```
func (s *Session) addMagnet(link string, opt *AddTorrentOptions) (*Torrent, err
// 1. 마그넷 링크 파싱
ma, err := magnet.New(link)
if err != nil {
return nil, newInputError(err)
}
// 2. 토렌트 객체 생성 (info = nil)
t, err := newTorrent2(
s, id, time.Now(), ma.InfoHash[:], sto, ma.Name, port,
s.parseTrackers(ma.Trackers, false),
```

```
ma.Peers,
nil, // info는 nil - 나중에 다운로드
nil, // bitfield도 nil
resumer.Stats{},
nil, // webseedSources
opt.StopAfterDownload,
opt.StopAfterMetadata,
false,
)
}
```

마그넷 링크 특징:

- `info` 필드가 nil로 시작
- 메타데이터를 피어로부터 다운로드해야 함
- 고정 피어 목록 (`x.pe` 파라미터) 지원

2. 토렌트 시작 프로세스 (torrent.start())

2.1 start() 함수 개요

```
func (t *torrent) start() {
// 이미 시작된 경우 무시
if  t.errC != nil {
return
```

```
}

t.log.Info("starting torrent")
t.errC = make(chan  error, 1)
t.portC = make(chan  int, 1)
t.lastError = nil
t.downloadSpeed = metrics.NewMeter()
t.uploadSpeed = metrics.NewMeter()

// 상태에 따른 분기 처리
if  t.info != nil {
if  t.pieces != nil {
if  t.bitfield != nil {
// 완전한 상태: 바로 다운로드 시작
t.addFixedPeers()
t.startAcceptor()
t.startAnnouncers()
t.startPieceDownloaders()
} else {
// 검증 필요
t.startVerifier()
}
} else {
```

```
// 파일 할당 필요
t.startAllocator()
}
} else {
// 메타데이터 다운로드 필요 (마그넷 링크)
t.addFixedPeers()
t.startAcceptor()
t.startAnnouncers()
t.startInfoDownloaders()
}
}
```

2.2 상태별 시작 프로세스

2.2.1 파일 할당 프로세스 (startAllocator)

새로운 토렌트의 경우 디스크에 파일을 할당해야 합니다:

```
func (t *torrent) startAllocator() {
if  t.allocator != nil {
t.crash("allocator exists")
}
t.allocator = allocator.New()
go  t.allocator.Run(t.info, t.storage, t.allocatorProgressC, t.allocatorResultC)
```

```
}
```

할당 프로세스:

1. `allocator.New()` : 새 할당자 생성
2. `allocator.Run()` : 별도 고루틴에서 파일 할당 실행
3. 진행상황은 `allocatorProgressC` 로 전달
4. 완료 시 `allocatorResultC` 로 결과 전달

2.2.2 검증 프로세스 (startVerifier)

기존 파일이 있는 경우 무결성 검증을 수행합니다:

```go
func (t *torrent) startVerifier() {
if t.verifier != nil {
t.crash("verifier exists")
}
if len(t.pieces) == 0 {
t.crash("zero length pieces")
}
t.verifier = verifier.New()
go t.verifier.Run(t.pieces, t.verifierProgressC, t.verifierResultC)
}
```

검증 프로세스:

1. 각 피스의 SHA-1 해시 계산
2. 메타정보의 해시와 비교

3. 유효한 피스는 bitfield에 마킹

4. 손상된 피스는 재다운로드 대상으로 표시

2.3 네트워크 구성요소 시작

2.3.1 연결 수락자 시작 (startAcceptor)

```
func (t *torrent) startAcceptor() {
if  t.acceptor != nil {
return
}
ip := net.ParseIP(t.session.config.Host)
listener, err := net.ListenTCP("tcp4", &net.TCPAddr{IP: ip, Port: t.port})
if  err != nil {
t.log.Warningf("cannot listen port %d: %s", t.port, err)
} else {
t.log.Info("Listening peers on tcp://" + listener.Addr().String())
t.port = listener.Addr().(*net.TCPAddr).Port
t.portC <- t.port
t.acceptor = acceptor.New(listener, t.incomingConnC, t.log)
go  t.acceptor.Run()
}
}
```

수락자 역할:

- 지정된 포트에서 TCP 연결 대기
- 새 연결을 `incomingConnC` 채널로 전달
- 별도 고루틴에서 실행

2.3.2 트래커 공지자 시작 (startAnnouncers)

```
func (t *torrent) startAnnouncers() {
// 기존 공지자가 없는 경우 새로 생성
if len(t.announcers) == 0 {
for _, tr := range t.trackers {
t.startNewAnnouncer(tr)
}
}
// DHT 공지자 시작 (프라이빗 토렌트가 아닌 경우)
if t.dhtAnnouncer == nil && t.session.config.DHTEnabled &&
(t.info == nil || !t.info.Private) {
t.dhtAnnouncer = announcer.NewDHTAnnouncer()
go t.dhtAnnouncer.Run(t.announceDHT,
t.session.config.DHTAnnounceInterval,
t.session.config.DHTMinAnnounceInterval,
t.log)
}
}
```

공지자 종류:

1. **HTTP/UDP 트래커 공지자**: 주기적으로 트래커에 상태 보고
2. **DHT 공지자**: DHT 네트워크에 토렌트 존재 알림

3. 메타데이터 다운로드 프로세스

3.1 InfoDownloader 개요

마그넷 링크로 추가된 토렌트는 메타데이터가 없으므로 피어로부터 다운로드해야 합니다.

```
type InfoDownloader struct {
Peer Peer
Bytes []byte
blocks []block
pending int // 진행 중인 요청 수
nextBlockIndex uint32
}
```

3.2 메타데이터 다운로드 시작

```
func (t *torrent) startInfoDownloaders() {
if t.info != nil {
return // 이미 메타데이터가 있음
}
// 병렬 다운로드 수 제한
```

```
for len(t.infoDownloaders)-len(t.infoDownloadersSnubbed) <
t.session.config.ParallelMetadataDownloads {
id := t.nextInfoDownload()
if id == nil {
break
}
pe := id.Peer.(*peer.Peer)
t.infoDownloaders[pe] = id
id.RequestBlocks(t.maxAllowedRequests(pe))
pe.ResetSnubTimer()
}
}
```

3.3 메타데이터 블록 처리

메타데이터는 16KB 블록으로 분할되어 다운로드됩니다:

```
func (d *InfoDownloader) GotBlock(index uint32, data []byte) error {
if index >= uint32(len(d.blocks)) {
return fmt.Errorf("peer sent invalid metadata piece index: %q", index)
}
b := &d.blocks[index]
if !b.requested {
return fmt.Errorf("peer sent unrequested index for metadata message: %q", index
```

```go
}
if uint32(len(data)) != b.size {
return fmt.Errorf("peer sent invalid size for metadata message: %q", len(data))
}
d.pending--
begin := index * blockSize
end := begin + b.size
copy(d.Bytes[begin:end], data)
return nil
}
```

메타데이터 다운로드 특징:

- 16KB 고정 블록 크기 (`blockSize = 16 * 1024`)
- 마지막 블록은 크기가 다를 수 있음
- 요청하지 않은 블록은 거부
- 크기가 맞지 않는 블록은 거부

3.4 메타데이터 완료 처리

메타데이터 다운로드가 완료되면 토렌트는 정상적인 다운로드 상태로 전환됩니다:

```go
func (t *torrent) handleInfoDownloaderDone(id *infodownloader.InfoDownloader) {
// 메타데이터 파싱 및 검증
info, err := metainfo.NewInfo(id.Bytes)
if err != nil {
```

```
t.log.Error("invalid metadata received:", err)
return
}
// 토렌트 정보 업데이트
t.info = info
t.piecePool = bufferpool.New(int(t.info.PieceLength))
// 파일 할당 시작
t.startAllocator()
// StopAfterMetadata 옵션 처리
if  t.stopAfterMetadata {
t.stopAndSetStoppedOnMetadata()
}
}
```

4. 토렌트 중지 및 정리 프로세스

4.1 stop() 함수 개요

```
func (t *torrent) stop(err  error) {
s := t.status()
if  s == Stopping || s == Stopped {
return  // 이미 중지 중이거나 중지됨
}
```

```
t.log.Info("stopping torrent")

t.lastError = err

if  err != nil && err != errClosed {

t.log.Error(err)

}


// 각 구성요소 순차적 중지

t.stopAcceptor()

t.stopPeers()

t.stopPiecedownloaders()

t.stopInfoDownloaders()

t.stopWebseedDownloads()


// 비트필드 저장

if  t.bitfield != nil {

_ = t.writeBitfield()

}


// 공지자 중지 및 Stopped 이벤트 공지

announcers := t.announcers

t.stopPeriodicalAnnouncers()
```

```
// 데이터 파일 닫기
t.closeData()
t.stopAllocator()
t.stopVerifier()

// 핸드셰이커 중지
t.stopOutgoingHandshakers()
t.stopIncomingHandshakers()

// 속도 측정 중지
t.resetSpeeds()

// Stopped 이벤트 공지자 시작
t.startStoppedEventAnnouncer(announcers)
}
```

4.2 구성요소별 중지 프로세스

4.2.1 피어 연결 중지

```
func (t *torrent) stopPeers() {
t.log.Debugln("closing peer connections")
for  p := range  t.peers {
t.closePeer(p)
```

```
}
}

func (t *torrent) closePeer(pe *peer.Peer) {
// 피어 맵에서 제거
delete(t.peers, pe)
delete(t.incomingPeers, pe)
delete(t.outgoingPeers, pe)
// 다운로더 정리
t.closePieceDownloader(pe)
t.closeInfoDownloader(pe)
// 연결 종료
pe.Close()
}
```

4.2.2 데이터 파일 닫기

```
func (t *torrent) closeData() {
t.log.Debugln("closing open files")
for _, f := range t.files {
err := f.Storage.Close()
if err != nil {
t.log.Error(err)
}
```

```
}

// 메모리 정리

t.files = nil

t.pieces = nil

t.piecePicker = nil

t.bytesAllocated = 0

t.checkedPieces = 0

}
```

4.2.3 Stopped 이벤트 공지

토렌트 중지 시 트래커에 Stopped 이벤트를 알려야 합니다:

```
func (t *torrent) startStoppedEventAnnouncer(announcers []*announcer.PeriodicalA

// 공지한 적이 있는 트래커만 선별

trackers := make([]tracker.Tracker, 0, len(announcers))

for _, an := range announcers {

if an.HasAnnounced {

trackers = append(trackers, an.Tracker)

}

}

// Stopped 이벤트 공지자 생성

t.stoppedEventAnnouncer = announcer.NewStopAnnouncer(

trackers,
```

```
t.announcerFields(),
t.session.config.TrackerStopTimeout,
t.announcersStoppedC,
t.log)
go  t.stoppedEventAnnouncer.Run()
}
```

4.3 완료 후 처리

4.3.1 자동 중지 옵션

```
func (t *torrent) stopAndSetStoppedOnComplete() {
err := t.session.resumer.HandleStopAfterDownload(t.id)
if  err != nil {
t.log.Errorf("cannot write status to resume db: %s", err)
}
t.stop(nil)
}
```

자동 중지 조건:

- `StopAfterDownload` : 다운로드 완료 시 자동 중지
- `StopAfterMetadata` : 메타데이터 다운로드 완료 시 자동 중지

4.3.2 재시작 데이터 업데이트

중지 시 현재 상태를 데이터베이스에 저장하여 재시작 시 복원할 수 있도록 합니다:

```go
func (t *torrent) writeBitfield() error {
if  t.bitfield == nil {
return  nil
}
data := t.bitfield.Bytes()
return  t.session.resumer.WriteBitfield(t.id, data)
}
```

5. 토렌트 상태 전환

5.1 상태 정의

```go
const (
Stopped Status = iota
Allocating
Verifying
Downloading
Seeding
Stopping
)
```

5.2 상태 전환 조건

```
func (t *torrent) status() Status {
select {
case <-t.closeC:
return  Stopped
default:
}
if  t.errC == nil {
return  Stopped
}
if  t.stoppedEventAnnouncer != nil {
return  Stopping
}
if  t.allocator != nil {
return  Allocating
}
if  t.verifier != nil {
return  Verifying
}
if  t.completed {
return  Seeding
}
return  Downloading
```

```
}
```

6. 에러 처리 및 복구

6.1 에러 전파

토렌트에서 발생한 에러는 `errC` 채널을 통해 전파됩니다:

```
func (t *torrent) crash(msg  string) {
err := errors.New(msg)
t.log.Error(err)
select {
case  t.errC <- err:
default:
}
}
```

6.2 재시작 메커니즘

토렌트는 특정 조건에서 자동으로 재시작될 수 있습니다:

```
func (t *torrent) handleStopped() {
t.stoppedEventAnnouncer = nil
t.errC <- t.lastError
t.errC = nil
t.portC = nil
```

```
if  t.doVerify {

t.bitfield = nil

t.start() // 검증 후 재시작

} else {

t.log.Info("torrent has stopped")

}

}
```

7. 성능 최적화

7.1 리소스 관리

- **포트 풀링**: 사용 가능한 포트 범위에서 동적 할당
- **메모리 풀링**: 피스 버퍼 재사용으로 GC 부하 감소
- **연결 제한**: 최대 연결 수 제한으로 리소스 보호

7.2 동시성 처리

- **고루틴 분리**: 각 구성요소가 독립적인 고루틴에서 실행
- **채널 통신**: 고루틴 간 안전한 메시지 전달
- **논블로킹 작업**: 긴 작업은 별도 고루틴에서 처리

결론

Rain BitTorrent 클라이언트의 토렌트 라이프사이클은 명확하게 정의된 상태와 전환 조건을 가지고 있습니다. 각 단계는 독립적으로 처리되며, 에러 발생 시 적절한 복구 메커니즘을 제공합니다. 이러한

설계는 안정성과 성능을 모두 확보하면서도 복잡한 BitTorrent 프로토콜을 효과적으로 구현할 수 있게 합니다.

관련 문서

선행 지식

- 01-system-architecture.md - Session과 Torrent 구조 이해
- 02-data-flow-analysis.md - 토렌트 추가 데이터 흐름

관련 주제

- 06-03-error-handling-recovery-analysis.md - 토렌트 에러 처리 및 복구
- 06-02-concurrency-analysis.md - 토렌트 고루틴 관리

후속 학습

- 04-01-networking-packages-analysis.md - 피어 연결 구현 세부사항
- 05-01-dht-implementation-analysis.md - DHT 통합 과정
- 07-01-library-usage-guide.md - 토렌트 관리 API 사용법

> **네비게이션:** 이전: 데이터 흐름 분석 | 목차 | 다음: 네트워킹 패키지 분석